جامعة الإمام عبدالرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY
كلية علوم الحاسب وتقنية المعلومات
College of Computer Science and Information Technology

CIS 321 PROJECT MILESTONE (2): Mapped Tables Filled with data and SQL Query Statements

Project Title: Database for a Food Delivery Application

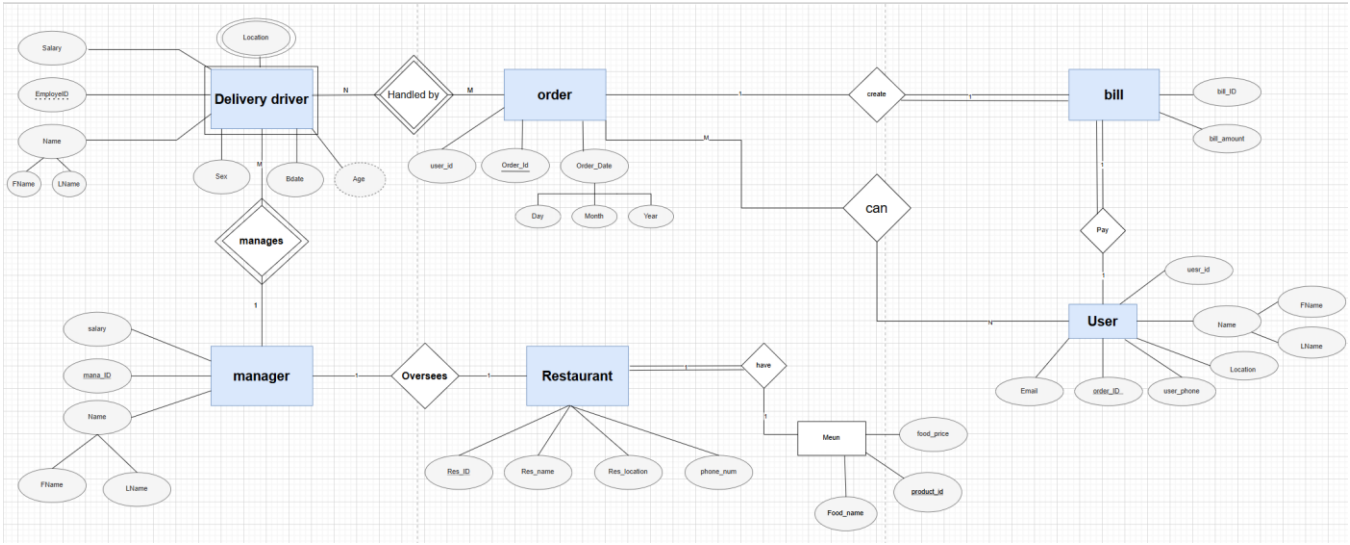
Section Number: F11 – G3

Date of Submission: Week 13, Sunday

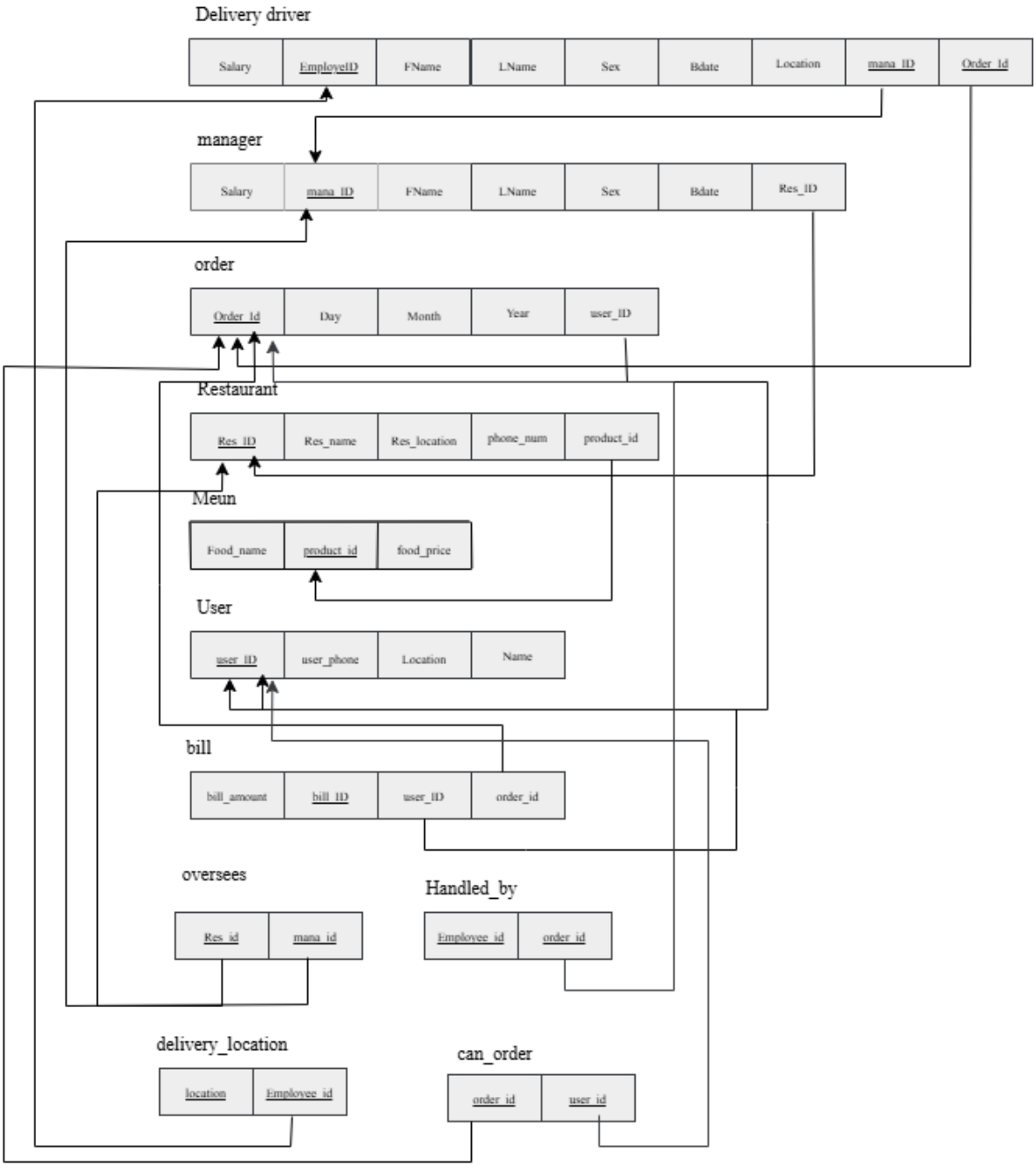
Student name:	ID
Nour Alkhames **(Leader)**	
Mashaal Abdullah	
Alaa Alshahrani	
Norah Al-Subaie	
Arwa Alqahtani	
Jana Alqahtani	

Table of Contents:

1.ER Diagram :


Salary
Location
EmployeeID
Delivery driver
N
Handled by
M
order
create
1
bill
bill_ID
Name
bill_amount
FName
LName
Sex
Bdate
Age
user_id
Order_Id
Order_Date
M
Day
Month
Year
can
Pay
manages
1
user_id
User
FName
Name
LName
Location
Email
order_ID
user_phone
salary
mana_ID
manager
Oversees
Restaurant
have
Name
FName
LName
Res_ID
Res_name
Res_location
phone_num
Meun
food_price
product_id
Food_name

2. MAPPING of your ER diagram into Database Schema:



3. The tables creating:

Delivery driver :

```
CREATE TABLE Delivery_driver (
    EmployeeID INT PRIMARY KEY,
    FName VARCHAR(20) NOT NULL,
    LName VARCHAR(20) NOT NULL,
    Sex CHAR(1) NOT NULL CHECK (Sex IN ('M', 'F')),
    Bdate DATE NOT NULL,
    Salary DECIMAL(10,2) NOT NULL CHECK (Salary >= 0),
    Location VARCHAR(100) NOT NULL
);
```

Manager:

```
CREATE TABLE Manager (
    Mana_ID INT PRIMARY KEY,
    FName VARCHAR(20) NOT NULL,
    LName VARCHAR(20) NOT NULL,
    Salary DECIMAL(10,2) NOT NULL CHECK (Salary >= 0)
);
```

Driver_Location:

```
CREATE TABLE Driver_Location (
    EmployeeID INT,
    Location VARCHAR(100) NOT NULL,
    PRIMARY KEY (EmployeeID, Location),
    FOREIGN KEY (EmployeeID) REFERENCES Delivery_driver(EmployeeID)
);
```

Restaurant :

```
CREATE TABLE Restaurant (
    Res_ID INT PRIMARY KEY,
    Res_name VARCHAR(100) NOT NULL UNIQUE,
    Res_location VARCHAR(100) NOT NULL,
    phone_num VARCHAR(10) NOT NULL
);
```

Oversees:

```
CREATE TABLE Oversees (
    Mana_ID INT  ,
    Res_ID INT ,
    FOREIGN KEY (Mana_ID) REFERENCES Manager(Mana_ID),
    FOREIGN KEY (Res_ID) REFERENCES Restaurant(Res_ID)
);
```

Menu:

```
CREATE TABLE Menu (
    product_ID INT PRIMARY KEY ,
    Food_name VARCHAR(100) NOT NULL,
    Food_price DECIMAL(10,2) NOT NULL CHECK (Food_price > 0),
    Res_ID INT , -- 1 restaurant has 1 menu (1:1 HAVE)
    FOREIGN KEY (Res_ID) REFERENCES Restaurant(Res_ID)
);
```

Users:

```
CREATE TABLE Users (
    User_ID INT PRIMARY KEY,
    FName VARCHAR(20) NOT NULL,
    LName VARCHAR(20) NOT NULL,
    Location VARCHAR(100) NOT NULL,
    user_phone VARCHAR(20) NOT NULL
);
```

Orders:

```
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Order_Year INT NOT NULL CHECK (Order_Year BETWEEN 2000 AND 2100),
    Order_Month INT NOT NULL CHECK (Order_Month BETWEEN 1 AND 12),
    Order_Day INT NOT NULL CHECK (Order_Day BETWEEN 1 AND 31)
);
```

Handled_by:

```
CREATE TABLE Handled_by (
    Order_ID INT,
    EmployeeID INT,
    PRIMARY KEY (Order_ID, EmployeeID),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
    FOREIGN KEY (EmployeeID) REFERENCES Delivery_driver(EmployeeID)
);

-- 10. Bill table
```

Bill:

```sql
CREATE TABLE Bill (
    Bill_ID INT PRIMARY KEY,
    Bill_Amount DECIMAL(10,2) NOT NULL CHECK (Bill_Amount > 0),
    Order_ID INT ,
    User_ID INT ,
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
    FOREIGN KEY (User_ID) REFERENCES Users(User_ID)
);
```

can_order:

```sql
-- 11. can_order relationship
CREATE TABLE can_order (
    User_ID INT,
    Order_ID INT,
    PRIMARY KEY (User_ID, Order_ID),
    FOREIGN KEY (User_ID) REFERENCES Users(User_ID),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
);
```

4. Fill Tables with 20 records:

Manager:

```sql
-- Inserting data into Manager table
INSERT INTO Manager (Mana_ID, FName, LName, Salary) VALUES
(1, 'Ali', 'Alsalem', 5000),
(2, 'Reem', 'Aljafri', 4500),
(3, 'Salem', 'Albreeki', 4700),
(4, 'Sara', 'Alghamdi', 4800),
(5, 'Nasser', 'Almutairi', 4600),
(6, 'Rami', 'Alnasser', 4700),
(7, 'Tariq', 'Alzahrani', 4900),
(8, 'Noor', 'Alomari', 4500),
(9, 'Maha', 'Alshaikh', 4200),
(10, 'Khalid', 'Alamri', 4600),
(11, 'Mohammed', 'Almazrouei', 4700),
(12, 'Joud', 'Albalawi', 4800),
(13, 'Layla', 'Alhussaini', 4900),
(14, 'Khaled', 'Alqasim', 5000),
(15, 'Sara', 'Alharbi', 5200),
(16, 'Amal', 'Almutairi', 4600),
(17, 'Zainab', 'Alkhathlan', 4800),
(18, 'Hassan', 'Alrajhi', 4700),
(19, 'Yousra', 'Aljarba', 4600),
(20, 'Tariq', 'Alhamad', 4500);
```

```sql
SELECT * FROM projectdb.manager;
```

Result Grid | Filter Rows: | Edit

Mana_ID	FName	LName	Salary
1	Ali	Alsalem	5000.00
2	Reem	Aljafri	4500.00
3	Salem	Albreeki	4700.00
4	Sara	Alghamdi	4800.00
5	Nasser	Almutairi	4600.00
6	Rami	Alnasser	4700.00
7	Tariq	Alzahrani	4900.00
8	Noor	Alomari	4500.00
9	Maha	Alshaikh	4200.00
10	Khalid	Alamri	4600.00
11	Moha...	Almazrouei	4700.00
12	Joud	Albalawi	4800.00
13	Layla	Alhussaini	4900.00
14	Khaled	Alqasim	5000.00
15	Sara	Alharbi	5200.00
16	Amal	Almutairi	4600.00
17	Zainab	Alkhathlan	4800.00
18	Hassan	Alrajhi	4700.00
19	Yousra	Aljarba	4600.00
20	Tariq	Alhamad	4500.00
NULL	NULL	NULL	NULL

Driver_Location:

```
INSERT INTO Driver_Location (EmployeeID, Location) VALUES
(1, 'Riyadh'),
(2, 'Jeddah'),
(3, 'Dammam'),
(4, 'Khobar'),
(5, 'Makkah'),
(6, 'Riyadh'),
(7, 'Jeddah'),
(8, 'Dammam'),
(9, 'Khobar'),
(10, 'Makkah'),
(11, 'Riyadh'),
(12, 'Jeddah'),
(13, 'Dammam'),
(14, 'Khobar'),
(15, 'Makkah'),
(16, 'Riyadh'),
(17, 'Jeddah'),
(18, 'Dammam'),
(19, 'Khobar'),
(20, 'Makkah');
```

EmployeeID	Location
1	Riyadh
2	Jeddah
3	Dammam
4	Khobar
5	Makkah
6	Riyadh
7	Jeddah
8	Dammam
9	Khobar
10	Makkah
11	Riyadh
12	Jeddah
13	Dammam
14	Khobar
15	Makkah
16	Riyadh
17	Jeddah
18	Dammam
19	Khobar
20	Makkah

Menu:

```
-- Inserting data into Menu table
INSERT INTO Menu (produect_ID, Food_name, Food_price, Res_ID) VALUES
(1, 'Margherita Pizza', 25, 1), (2, 'Pasta Bolognese', 20, 1), (3, 'Pepperoni Pizza', 22, 2),
(4, 'Cheese Breadsticks', 15, 2), (5, 'Veggie Pizza', 18, 3), (6, 'Chicken Wings', 16, 3),
(7, 'Cheeseburger', 18, 4), (8, 'French Fries', 8, 4), (9, 'Pepperoni Pizza', 25, 5),
(10, 'Bffalo Wings', 18, 5), (11, 'Zinger Sandwich', 17, 6), (12, 'Fried Chicken', 20, 7),
(13, 'Big Burger', 22, 8), (14, 'Chicken Sandwich', 18, 9), (15, 'Whopper Burger', 24, 10),
(16, 'Crispy Chicken', 21, 11), (17, 'BBQ Ribs', 27, 12), (18, 'Pasta Carbonara', 23, 13),
(19, 'Cheese Fries', 11, 14), (20, 'Garlic Bread', 10, 15);
```

```
SELECT * FROM projectdb.menu;
```

Result Grid | Filter Rows: | Edit:

produect_ID	Food_name	Food_price	Res_ID
1	Margherita Pizza	25.00	1
2	Pasta Bolognese	20.00	1
3	Pepperoni Pizza	22.00	2
4	Cheese Breadsticks	15.00	2
5	Veggie Pizza	18.00	3
6	Chicken Wings	16.00	3
7	Cheeseburger	18.00	4
8	French Fries	8.00	4
9	Pepperoni Pizza	25.00	5
10	Bffalo Wings	35.00	5
11	Zinger Sandwich	17.00	6
12	Fried Chicken	20.00	7
13	Big Burger	22.00	8
14	Chicken Sandwich	18.00	9
15	Whopper Burger	24.00	10
16	Crispy Chicken	21.00	11
17	BBQ Ribs	27.00	12
18	Pasta Carbonara	23.00	13
19	Cheese Fries	11.00	14
20	Garlic Bread	10.00	15
NULL	NULL	NULL	NULL

Orders:

```
238    -- Inserting data into Orders table
239 •  INSERT INTO Orders (Order_ID, Order_Year, Order_Month, Order_Day) VALUES
240    (1, 2025, 4, 15), (2, 2025, 4, 16), (3, 2025, 4, 17), (4, 2025, 4, 18), (5, 2025, 4, 19),
241    (6, 2025, 4, 20), (7, 2025, 4, 21), (8, 2025, 4, 22), (9, 2025, 4, 23), (10, 2025, 4, 24),
242    (11, 2025, 4, 25), (12, 2025, 4, 26), (13, 2025, 4, 27), (14, 2025, 4, 28), (15, 2025, 4, 29),
243    (16, 2025, 4, 30), (17, 2025, 5, 1), (18, 2025, 5, 2), (19, 2025, 5, 3), (20, 2025, 5, 4);
```

```
1 •  SELECT * FROM projectdb.orders;
```

Result Grid | Filter Rows: | Edit:

Order_ID	Order_Year	Order_Month	Order_Day
1	2025	4	15
2	2025	4	16
3	2025	4	17
4	2025	4	18
5	2025	4	19
6	2025	4	20
7	2025	4	21
8	2025	4	22
9	2025	4	23
10	2025	4	24
11	2025	4	25
12	2025	4	26
13	2025	4	27
14	2025	4	28
15	2025	4	29
16	2025	4	30
17	2025	5	1
18	2025	5	2
19	2025	5	3
20	2025	5	4
21	2025	5	5
NULL	NULL	NULL	NULL

Handled_by:

```
-- Inserting data into Handled_by table
INSERT INTO Handled_by (Order_ID, EmployeeID) VALUES
(1, 1), (2, 2), (3, 3), (4, 4), (5, 5),
(6, 6), (7, 7), (8, 8), (9, 9), (10, 10),
(11, 11), (12, 12), (13, 13), (14, 14), (15, 15),
(16, 16), (17, 17), (18, 18), (19, 19), (20, 20);
```

```
1 •  SELECT * FROM projectdb.handled_by;
```

Result Grid | Filter Rows: | Edit:

Order_ID	EmployeeID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
NULL	NULL

Oversees:

```sql
-- Inserting data into Oversees table
INSERT INTO Oversees (Mana_ID, Res_ID) VALUES
(1, 1), -- Manager 1 oversee Restaurant 1
(2, 2), -- Manager 2 oversees Restaurant 2
(3, 3), -- Manager 3 oversees Restaurant 3
(4, 4), -- Manager 4 oversees Restaurant 4
(5, 5), -- Manager 5 oversees Restaurant 5
(6, 6), -- Manager 6 oversees Restaurant 6
(1, 7), -- Manager 1 oversees Restaurant 7
(2, 8), -- Manager 2 oversees Restaurant 8
(3, 9), -- Manager 3 oversees Restaurant 9
(4, 10), -- Manager 4 oversees Restaurant 10
(5, 11), -- Manager 5 oversees Restaurant 11
(6, 12), -- Manager 6 oversees Restaurant 12
(1, 13), -- Manager 1 oversees Restaurant 13
(2, 14), -- Manager 2 oversees Restaurant 14
(3, 15), -- Manager 3 oversees Restaurant 15
(4, 16), -- Manager 4 oversees Restaurant 16
(5, 17), -- Manager 5 oversees Restaurant 17
(6, 18), -- Manager 6 oversees Restaurant 18
(1, 19), -- Manager 1 oversees Restaurant 19
(2, 20); -- Manager 2 oversees Restaurant 20
```

```sql
SELECT * FROM projectdb.oversees;
```

Result Grid | Filter Rows: | Expor

Mana_ID	Res_ID
1	1
2	2
3	3
4	4
5	5
6	6
1	7
2	8
3	9
4	10
5	11
6	12
1	13
2	14
3	15
4	16
5	17
6	18
1	19
2	20

Can_order:

```sql
SELECT * FROM projectdb.can_order;
```

```sql
-- Inserting data into can_order table
INSERT INTO can_order (User_ID, Order_ID) VALUES
(1, 1), (2, 2), (3, 3), (4, 4), (5, 5),
(6, 6), (7, 7), (8, 8), (9, 9), (10, 10),
(11, 11), (12, 12), (13, 13), (14, 14), (15, 15),
(16, 16), (17, 17), (18, 18), (19, 19), (20, 20);
```

Result Grid | Filter Rows: | Edit:

User_ID	Order_ID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
21	21
NULL	NULL

Bill:

```sql
INSERT INTO Bill (Bill_ID, Bill_Amount, Order_ID, User_ID) VALUES
(1, 45.5, 1, 1),
(2, 60.0, 2, 2),
(3, 58.0, 3, 3),
(4, 72.0, 4, 4),
(5, 68.0, 5, 5),
(6, 55.0, 6, 6),
(7, 65.0, 7, 7),
(8, 62.0, 8, 8),
(9, 70.0, 9, 9),
(10, 75.0, 10, 10),
(11, 80.0, 11, 11),
(12, 85.0, 12, 12),
(13, 90.0, 13, 13),
(14, 95.0, 14, 14),
(15, 100.0, 15, 15),
(16, 105.0, 16, 16),
(17, 110.0, 17, 17),
(18, 115.0, 18, 18),
(19, 120.0, 19, 19),
(20, 125.0, 20, 20);
```

```sql
SELECT * FROM projectdb.bill;
```

Result Grid | Filter Rows:

Bill_ID	Bill_Amount	Order_ID	User_ID
1	45.50	1	1
2	60.00	2	2
3	58.00	3	3
4	72.00	4	4
5	68.00	5	5
6	55.00	6	6
7	65.00	7	7
8	62.00	8	8
9	70.00	9	9
10	75.00	10	10
11	80.00	11	11
12	85.00	12	12
13	90.00	13	13
14	95.00	14	14
15	100.00	15	15
16	105.00	16	16
17	110.00	17	17
18	115.00	18	18
19	120.00	19	19
20	125.00	20	20

Users:

```sql

-- Inserting data into Users table
INSERT INTO Users (User_ID, FName, LName, Location, user_phone) VALUES
(1, 'Mohammed', 'Alali', 'Riyadh', '501234567'),
(2, 'Sarah', 'Altamimi', 'Jeddah', '559876543'),
(3, 'Sami', 'Alotaibi', 'Dammam', '534567890'),
(4, 'Noor', 'Alzahra', 'Khobar', '523456789'),
(5, 'Fayez', 'Almalki', 'Makkah', '509876543'),
(6, 'Khalid', 'Alfahad', 'Riyadh', '521234567'),
(7, 'Reem', 'Alkhaldi', 'Jeddah', '539876543'),
(8, 'Tariq', 'Alzahrani', 'Dammam', '523456789'),
(9, 'Omar', 'Alhajri', 'Khobar', '509876543'),
(10, 'Layla', 'Aljuhani', 'Makkah', '510876543'),
(11, 'Rana', 'Alqasim', 'Riyadh', '511234567'),
(12, 'Ahmed', 'Almohammed', 'Jeddah', '512345678'),
(13, 'Maha', 'Alrashed', 'Dammam', '513456789'),
(14, 'Lina', 'Alghamdi', 'Khobar', '514567890'),
(15, 'Khaled', 'Alnajjar', 'Makkah', '515678901'),
(16, 'Reem', 'Alamri', 'Riyadh', '516789012'),
(17, 'Rasha', 'Almutairi', 'Jeddah', '517890123'),
(18, 'Mohammad', 'Alqasim', 'Dammam', '518901234'),
(19, 'Faisal', 'Alzahrani', 'Khobar', '519012345'),
(20, 'Yousef', 'Alrashid', 'Makkah', '520123456');

```

```sql
SELECT * FROM projectdb.users;
```

Result Grid | Filter Rows: | Edit:

User_ID	FName	LName	Location	user_phone
1	Mohammed	Alali	جدة	501234567
2	Sarah	Altamimi	Jeddah	559876543
3	Sami	Alotaibi	Dammam	534567890
4	Noor	Alzahra	Khobar	523456789
5	Fayez	Almalki	Makkah	509876543
6	Khalid	Alfahad	Riyadh	521234567
7	Reem	Alkhaldi	Jeddah	539876543
8	Tariq	Alzahrani	Dammam	523456789
9	Omar	Alhajri	Khobar	509876543
10	Layla	Aljuhani	Makkah	510876543
11	Rana	Alqasim	Riyadh	511234567
12	Ahmed	Almoha...	Jeddah	512345678
13	Maha	Alrashed	Dammam	513456789
14	Lina	Alghamdi	Khobar	514567890
15	Khaled	Alnajjar	Makkah	515678901
16	Reem	Alamri	Riyadh	516789012
17	Rasha	Almutairi	Jeddah	517890123
18	Mohammad	Alqasim	Dammam	518901234
19	Faisal	Alzahrani	Khobar	519012345
20	Yousef	Alrashid	Makkah	520123456
21	Hassan	Alrashid	Riyadh	530123456
NULL	NULL	NULL	NULL	NULL

Restaurant:

```sql
-- Inserting data into Restaurant table
INSERT INTO Restaurant (Res_ID, Res_name, Res_location, phone_num) VALUES
(1, 'Maestro', 'Riyadh', '112345678'),
(2, 'Domino''s', 'Jeddah', '123456789'),
(3, 'Little Caesars', 'Dammam', '134567890'),
(4, 'McDonald''s', 'Khobar', '145678901'),
(5, 'Pizza Hut', 'Makkah', '156789012'),
(6, 'TGI Fridays', 'Riyadh', '0167891234'),
(7, 'Hardee''s', 'Jeddah', '0178905432'),
(8, 'Burger King', 'Dammam', '0189016543'),
(9, 'Papa John''s', 'Khobar', '0190127654'),
(10, 'KFC', 'Makkah', '0201238765'),
(11, 'Starbucks', 'Riyadh', '0212349876'),
(12, 'Shake Shack', 'Jeddah', '0223451098'),
(13, 'Subway', 'Dammam', '0234562109'),
(14, 'Chili''s', 'Khobar', '0245673210'),
(15, 'Five Guys', 'Makkah', '0256784321'),
(16, 'Popeyes', 'Riyadh', '0267895432'),
(17, 'Wingstop', 'Jeddah', '0278906543'),
(18, 'Carls Jr.', 'Dammam', '0289017654'),
(19, 'Pizza Express', 'Khobar', '0290128765'),
(20, 'Dominos Pizza', 'Makkah', '0301239876');
```

```sql
SELECT * FROM projectdb.restaurant;
```

Result Grid | Filter Rows: | Edit:

Res_ID	Res_name	Res_location	phone_num
1	Maestro	Riyadh	112345678
2	Domino's	Jeddah	123456789
3	Little Caesars	Dammam	134567890
4	McDonald's	Khobar	145678901
5	Pizza Hut	Makkah	156789012
6	TGI Fridays	Riyadh	0167891234
7	Hardee's	Jeddah	0178905432
8	Burger King	Dammam	0189016543
9	Papa John's	Khobar	0190127654
10	KFC	Makkah	0201238765
11	Starbucks	Riyadh	0212349876
12	Shake Shack	Jeddah	0223451098
13	Subway	Dammam	0234562109
14	Chili's	Khobar	0245673210
15	Five Guys	Makkah	0256784321
16	Popeyes	Riyadh	0267895432
17	Wingstop	Jeddah	0278906543
18	Carls Jr.	Dammam	0289017654
19	Pizza Express	Khobar	0290128765
20	Dominos Pizza	Makkah	0301239876
NULL	NULL	NULL	NULL

Delivery_driver:

```sql
INSERT INTO Delivery_driver (EmployeeID, FName, LName, Sex, Bdate, Salary, Mana_ID) VALUES
(1, 'Mohammed', 'Alsaudi', 'M', '1985-08-15', 3000, 1),
(2, 'Sarah', 'Alotaibi', 'F', '1990-04-22', 2800, 2),
(3, 'Ahmed', 'Alhashemi', 'M', '1982-12-01', 3200, 3),
(4, 'Abdullah', 'Alshammari', 'M', '1987-01-10', 3100, 1),
(5, 'Jessica', 'Almari', 'F', '1992-06-30', 2900, 2),
(6, 'Khalid', 'Altamimi', 'M', '1993-05-12', 3100, 3),
(7, 'Daniel', 'Alqosaibi', 'M', '1986-11-25', 3300, 1),
(8, 'Fatima', 'Alharbi', 'F', '1991-02-18', 2800, 2),
(9, 'Yusuf', 'Alzaabi', 'M', '1988-09-14', 3500, 3),
(10, 'Aisha', 'Alhusseini', 'F', '1994-07-08', 3200, 1),
(11, 'Rania', 'Alhajri', 'F', '1990-03-10', 3200, 4),
(12, 'Ali', 'Alzahrani', 'M', '1985-05-20', 3400, 5),
(13, 'Layla', 'Almutairi', 'F', '1994-11-22', 3100, 6),
(14, 'Sami', 'Alghamdi', 'M', '1989-07-12', 3300, 4),
(15, 'Amir', 'Alshaikh', 'M', '1992-09-15', 3000, 5),
(16, 'Mona', 'Alzahrani', 'F', '1986-08-05', 3100, 6),
(17, 'Jamal', 'Alotaibi', 'M', '1988-12-25', 2800, 4),
(18, 'Fahad', 'Alshammari', 'M', '1987-06-10', 3500, 5),
(19, 'Khalid', 'Alraddadi', 'M', '1991-11-20', 3400, 6),
(20, 'Noura', 'Alfahad', 'F', '1993-04-11', 3300, 4);
```

EmployeeID	FName	LName	Sex	Bdate	Salary	Mana_ID
1	Mohammed	Alsaudi	M	1985-08-15	3000.00	1
2	Sarah	Alotaibi	F	1990-04-22	2800.00	2
3	Ahmed	Alhashemi	M	1982-12-01	3200.00	3
4	Abdullah	Alshammari	M	1987-01-10	3100.00	1
5	Jessica	Almari	F	1992-06-30	2900.00	2
6	Khalid	Altamimi	M	1993-05-12	3100.00	3
7	Daniel	Alqosaibi	M	1986-11-25	3300.00	1
8	Fatima	Alharbi	F	1991-02-18	2800.00	2
9	Yusuf	Alzaabi	M	1988-09-14	3500.00	3
10	Aisha	Alhusseini	F	1994-07-08	3200.00	1
11	Rania	Alhajri	F	1990-03-10	3200.00	4
12	Ali	Alzahrani	M	1985-05-20	3400.00	5
13	Layla	Almutairi	F	1994-11-22	3100.00	6
14	Sami	Alghamdi	M	1989-07-12	3300.00	4
15	Amir	Alshaikh	M	1992-09-15	3000.00	5
16	Mona	Alzahrani	F	1986-08-05	3100.00	6
17	Jamal	Alotaibi	M	1988-12-25	2800.00	4
18	Fahad	Alshammari	M	1987-06-10	3500.00	5
19	Khalid	Alraddadi	M	1991-11-20	3400.00	6
20	Noura	Alfahad	F	1993-04-11	3300.00	4

5. SQL Query Statements :

Insert:

```
290    -- Orders table
291 •  INSERT INTO Orders (Order_ID, Order_Year, Order_Month, Order_Day) VALUES
292    (22, 2025, 5, 5);
```

Output

Action Output

#	Time	Action	Message
1	17:10:20	INSERT INTO Users (User_ID, FName, LName, Location, user_phone) VALUES (22, 'Hassan', 'Arashid', 'Riyad...	1 row(s) affected
2	17:10:52	SELECT * FROM projectdb.users LIMIT 0, 1000	22 row(s) returned
3	17:11:30	INSERT INTO Orders (Order_ID, Order_Year, Order_Month, Order_Day) VALUES (21, 2025, 5, 5)	Error Code: 1062. Duplicate entry '21' for key 'orders.PRIMARY'
4	17:11:50	INSERT INTO Orders (Order_ID, Order_Year, Order_Month, Order_Day) VALUES (22, 2025, 5, 5)	1 row(s) affected

```
1 •  SELECT * FROM projectdb.orders;
```

Result Grid | Filter Rows:

Order_ID	Order_Year	Order_Month	Order_Day
1	2025	4	15
2	2025	4	16
3	2025	4	17
4	2025	4	18
5	2025	4	19
6	2025	4	20
7	2025	4	21
8	2025	4	22
9	2025	4	23
10	2025	4	24
11	2025	4	25
12	2025	4	26
13	2025	4	27
14	2025	4	28
15	2025	4	29
16	2025	4	30
17	2025	5	1
18	2025	5	2
19	2025	5	3
20	2025	5	4
21	2025	5	5
22	2025	5	5
NULL	NULL	NULL	NULL

Update:

```
319 •  UPDATE Menu
320    SET Food_price = 35
321    WHERE product_ID = 10;
322
```

Output

Action Output

#	Time	Action
21	17:30:22	SELECT User_ID, SUM(Bill_Amount) AS Total_Paid FROM Bill GROUP BY User_ID LIMIT 0, 1000
22	17:32:10	UPDATE Menu SET Food_price = 35 WHERE product_ID = 10

```
1 •  SELECT * FROM projectdb.menu;
```

Result Grid | Filter Rows: | Edit:

product_ID	Food_name	Food_price	Res_ID
1	Margherita Pizza	25.00	1
2	Pasta Bolognese	20.00	1
3	Pepperoni Pizza	22.00	2
4	Cheese Breadsticks	15.00	2
5	Veggie Pizza	18.00	3
6	Chicken Wings	16.00	3
7	Cheeseburger	18.00	4
8	French Fries	8.00	4
9	Pepperoni Pizza	25.00	5
10	Bffalo Wings	35.00	5
11	Zinger Sandwich	17.00	6
12	Fried Chicken	20.00	7
13	Big Burger	22.00	8
14	Chicken Sandwich	18.00	9
15	Whopper Burger	24.00	10
16	Crispy Chicken	21.00	11
17	BBQ Ribs	27.00	12
18	Pasta Carbonara	23.00	13
19	Cheese Fries	11.00	14
20	Garlic Bread	10.00	15
NULL	NULL	NULL	NULL

Delete:

Befor delete:

```
1 •  SELECT * FROM projectdb.bill;
```

Result Grid | Filter Rows: | Edit: | Expor

Bill_ID	Bill_Amount	Order_ID	User_ID
1	45.50	1	1
2	60.00	2	2
3	58.00	3	3
4	72.00	4	4
5	68.00	5	5
6	55.00	6	6
7	65.00	7	7
8	62.00	8	8
9	70.00	9	9
10	75.00	10	10
11	80.00	11	11
12	85.00	12	12
13	90.00	13	13
14	95.00	14	14
15	100.00	15	15
16	105.00	16	16
17	110.00	17	17
18	115.00	18	18
19	120.00	19	19
20	125.00	20	20
21	88.50	21	21
NULL	NULL	NULL	NULL

After delete :

```
323
324 •  DELETE FROM Bill WHERE Order_ID = 1;
325
326
327
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
2	22:34:41	SELECT * FROM projectdb.orders LIMIT 0, 1000	22 row(s) returned	0.187 sec / 0.000 sec
3	22:35:00	SELECT * FROM projectdb.orders LIMIT 0, 1000	22 row(s) returned	0.000 sec / 0.000 sec
4	22:36:17	DELETE FROM Orders WHERE Order_ID = 1	Error Code: 1451. Cannot delete or update a pare...	2.672 sec
5	22:36:54	DELETE FROM Bill WHERE Order_ID = 1	1 row(s) affected	2.812 sec

```
1 •  SELECT * FROM projectdb.bill;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

Bill_ID	Bill_Amount	Order_ID	User_ID
2	60.00	2	2
3	58.00	3	3
4	72.00	4	4
5	68.00	5	5
6	55.00	6	6
7	65.00	7	7
8	62.00	8	8
9	70.00	9	9
10	75.00	10	10
11	80.00	11	11
12	85.00	12	12
13	90.00	13	13
14	95.00	14	14
15	100.00	15	15
16	105.00	16	16
17	110.00	17	17
18	115.00	18	18
19	120.00	19	19
20	125.00	20	20
21	88.50	21	21
NULL	NULL	NULL	NULL

6. 15 SQL Queries for Restaurant Data Analysis and Management

1. **View all records from the Bill table**

```
42
43      -- 1. Select all data from the Bill table
44 •    SELECT * FROM Bill;
45
46
47 •
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Bill_ID	Bill_Amount	Order_ID	User_ID
1	45.50	1	1
2	60.00	2	2
3	58.00	3	3
4	72.00	4	4
5	68.00	5	5
6	55.00	6	6

2. **Total bill amount per user**

```
346     -- 2. Select User_ID and the total Bill_Amount for each User_ID
347 •   SELECT User_ID, SUM(Bill_Amount) AS Total_Paid
348     FROM Bill
349     GROUP BY User_ID;
350
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

User_ID	Total_Paid
1	45.50
2	60.00
3	58.00
4	72.00
5	68.00
6	55.00
7	65.00
8	62.00

3. **View menu items for restaurant with ID = 1**

```
350
351     -- 3. Select all data from the Menu table where Res_ID = 1
352 •   SELECT * FROM Menu
353     WHERE Res_ID = 1;
354
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Menu_ID	Food_name	Food_price	product_id	Res_ID
NULL	NULL	NULL	NULL	NULL

4. Union of User_IDs from Users and Bill

```
355     -- 4. Select User_ID from Users and Bill using UNION
356 •  SELECT User_ID FROM Users
357     UNION
358     SELECT User_ID FROM Bill;
359
360
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

User_ID
1
2
3
4
5
6

5. High-salary drivers and managers

```
50
51     -- 5.  Select FName, LName from Delivery_driver and Manager using UNION
52 •  SELECT FName, LName
53     FROM Delivery_driver
54     WHERE Salary > 3000
55     UNION
56     SELECT FName, LName
57     FROM Manager
58     WHERE Salary > 4500;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

FName	LName
Ahmed	Alhashemi
Abdullah	Alshammari
Khalid	Altamimi
Daniel	Alqosaibi
Yusuf	Alzaabi
Aisha	Alhusseini

6. Distinct restaurant locations starting with “Riyadh”

```
369
370     -- 6. Select DISTINCT Res_location from Restaurant where Res_location LIKE 'Riyadh%'
371 •  SELECT DISTINCT Res_location
372     FROM Restaurant
373     WHERE Res_location LIKE 'Riyadh%';
374
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Res_location
Riyadh

7. Delivery drivers managed by Manager 1

```
-- 7. Select delivery drivers managed by Manager 1

SELECT FName, LName
FROM Delivery_driver
WHERE Mana_ID = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FName	LName
Mohammed	Alsaudi
Abdullah	Alshammari
Daniel	Alqosaibi
Aisha	Alhusseini

8. Drivers with salary over 3000 sorted by salary

```
-- 8.  Display drivers with salary over 3000 and order by salary
SELECT FName, LName, Salary
FROM Delivery_driver
WHERE Salary > 3000
ORDER BY Salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FName	LName	Salary
Yusuf	Alzaabi	3500.00
Fahad	Alshammari	3500.00
Ali	Alzahrani	3400.00
Khalid	Alraddadi	3400.00
Daniel	Alqosaibi	3300.00
Sami	Alghamdi	3300.00
Noura	Alfahad	3300.00
Ahmed	Alhashemi	3200.00
Aisha	Alhusseini	3200.00
Rania	Alhajri	3200.00
Abdullah	Alshammari	3100.00
Khalid	Altamimi	3100.00
Layla	Almutairi	3100.00
Mona	Alzahrani	3100.00
Omar	Alsuwailem	3100.00

Delivery_driver 9

9. Count of menu items in restaurants with more than 10 items

```
-- 9.  Count menu items in restaurants with more than 2 foods
SELECT Res_name, COUNT(Menu.product_ID) AS Total_Foods
FROM Restaurant
INNER JOIN Menu ON Restaurant.Res_ID = Menu.Res_ID
GROUP BY Res_name
HAVING COUNT(Menu.product_ID) > 2;

-- 10. Select all data from Restaurant where Res_location = '
SELECT * FROM Restaurant
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Res_name	Total_Foods
Maestro	3

10. **All restaurant data where location is Riyadh**

```
395
396     -- 10. Select all data from Restaurant where Res_location = 'Riyadh'
397 •   SELECT * FROM Restaurant
398     WHERE Res_location = 'Riyadh';
399
400
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

Res_ID	Res_name	Res_location	phone_num
1	Restaurant A	Riyadh	501234567
NULL	NULL	NULL	NULL

11. Managers overseeing restaurants in Riyadh

```
401     -- 11. Show Managers overseeing restaurants in Riyadh
402 •   SELECT Manager.FName, Manager.LName
403     FROM Manager
404     INNER JOIN Oversees ON Manager.Mana_ID = Oversees.Mana_ID
405     INNER JOIN Restaurant ON Oversees.Res_ID = Restaurant.Res_ID
406     WHERE Restaurant.Res_location = 'Riyadh';
407
408
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FName	LName

12. Update food price in the Menu table

```
415     -- 12.
416 •   UPDATE Menu
417     SET Food_price = 20.50
418     WHERE product_ID = 1;
419
420 •   select * from Menu;
421
422     -- 13 Select foods with 'Chicken' in
```

Result Grid | Filter Rows: | Edit:

product_ID	Food_name	Food_price	Res_ID
1	Margherita Pizza	20.50	1
2	Pasta Bolognese	20.00	1

13. **Foods with “Chicken” in the name sorted by name**

```
413
414     -- 13 Select foods with 'Chicken' in the name and order by name
415 •   SELECT Food_name, Food_price
416     FROM Menu
417     WHERE Food_name LIKE '%Chicken%'
418     ORDER BY Food_name ASC;
419
420
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Food_name	Food_price

14. **User data with bill amounts using JOIN**

```
421     -- 14. Select data from Users and Bill using JOIN
422 •   SELECT Users.User_ID, Users.FName, Bill.Bill_Amount
423     FROM Users
424     INNER JOIN Bill ON Users.User_ID = Bill.User_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

User_ID	FName	Bill_Amount
3	Sami	58.00
4	Noor	72.00
5	Fayez	68.00
6	Khalid	55.00
7	Reem	65.00
8	Tariq	62.00
9	Omar	70.00
10	Layla	75.00
11	Rana	80.00
12	Ahmed	85.00
13	Maha	90.00
14	Lina	95.00
15	Khaled	100.00
16	Reem	105.00
17	Rasha	110.00
18	Mohammad	115.00
19	Faisal	120.00
20	Yousef	125.00
21	Hassan	88.50

15. Orders made after the year 2020

```
426     -- 15. Select Order_ID and Order_Year from Orders where Order_Year > 2020
427 •   SELECT Order_ID, Order_Year FROM Orders
428     WHERE Order_Year > 2020;
429
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

Order_ID	Order_Year
1	2025
2	2025
3	2025
4	2025
5	2025
6	2025
7	2025
8	2025
9	2025
10	2025
11	2025
12	2025
13	2025
14	2025
15	2025
16	2025
17	2025
18	2025
19	2025
20	2025

.

7. Database Functionality

Views:

This view will show the list of (userID_fullName_location_userPhone_Bill-ID_BillAmount) of each user.

```
1  •   CREATE OR REPLACE VIEW View_UserBillDetails AS
2      SELECT
3          u.User_ID,
4          CONCAT(u.FName, ' ', u.LName) AS FullName,
5          u.Location,
6          u.user_phone,
7          b.Bill_ID,
8          b.Bill_Amount
9      FROM
10         Users u
11     JOIN
12         Bill b ON u.User_ID = b.User_ID;  --  Correct JOIN clause
13
14 •   SELECT * FROM projectdb.view_userbilldetails;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

User_ID	FullName	Location	user_phone	Bill_ID	Bill_Amount
1	Mohammed Alali	Riyadh	501234567	1	45.50
2	Sarah Altamimi	Jeddah	559876543	2	60.00
3	Sami Alotaibi	Dammam	534567890	3	58.00
4	Noor Alzahra	Khobar	523456789	4	72.00
5	Fayez Almalki	Makkah	509876543	5	68.00

View the derived attributes:

```
631  -- derived attribute
632  CREATE OR REPLACE VIEW DriverWithAge AS
633  SELECT
634      EmployeeID,
635      CONCAT(FName, ' ', LName) AS DriverName,
636      Bdate,
637      TIMESTAMPDIFF(YEAR, Bdate, CURDATE()) AS Age
638  FROM
639      Delivery_driver;
640
641      SELECT * FROM DriverWithAge;
642
```

EmployeeID	DriverName	Bdate	Age
1	Mohammed Alsaudi	1985-08-15	39
2	Sarah Alotaibi	1990-04-22	35
3	Ahmed Alhashemi	1982-12-01	42
4	Abdullah Alshammari	1987-01-10	38
5	Jessica Almari	1992-06-30	32
6	Khalid Altamimi	1993-05-12	31
7	Daniel Alqosaibi	1986-11-25	38
8	Fatima Alharbi	1991-02-18	34
9	Yusuf Alzaabi	1988-09-14	36
10	Aisha Alhusseini	1994-07-08	30
11	Rania Alhajri	1990-03-10	35
12	Ali Alzahrani	1985-05-20	39
13	Layla Almutairi	1994-11-22	30
14	Sami Alghamdi	1989-07-12	35
15	Amir Alshaikh	1992-09-15	32
16	Mona Alzahrani	1986-08-05	38
17	Jamal Alotaibi	1988-12-25	36
18	Fahad Alshammari	1987-06-10	37
19	Khalid Alraddadi	1991-11-20	33
20	Noura Alfahad	1993-04-11	32

Triggers:

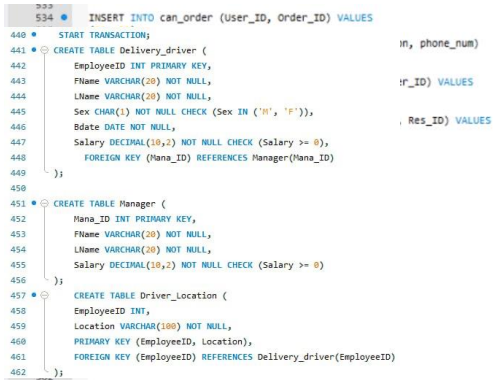
```
1  USE projectdb;
2  DROP TRIGGER IF EXISTS After_Bill_Insert;
3  DELIMITER //
4  CREATE TRIGGER After_Bill_Insert
5  AFTER INSERT ON Bill
6  FOR EACH ROW
7  BEGIN
8      INSERT INTO Log_table (Action_Description, Action_Time)
9      VALUES (CONCAT('Invoice number added: ', NEW.Bill_ID), NOW());
10 END //
11 DELIMITER ;
12
13 SELECT * from Bill order by Bill_ID;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

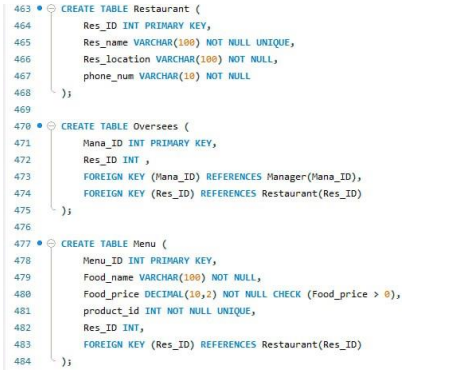
Bill_ID	Bill_Amount	Order_ID	User_ID
1	45.50	1	1
2	60.00	2	2
3	58.00	3	3
4	72.00	4	4
5	68.00	5	5

Transaction:

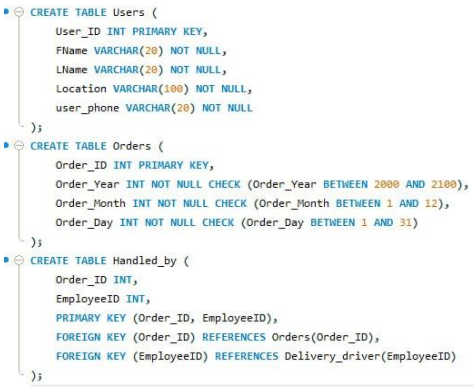
A transaction ensures that all operations (like adding, updating, or deleting data) are done correctly. If everything works, the changes are saved using COMMIT. If something goes wrong, ROLLBACK will undo all the changes.



```
INSERT INTO can_order (User_ID, Order_ID) VALUES
START TRANSACTION;
CREATE TABLE Delivery_driver (
    EmployeeID INT PRIMARY KEY,
    FName VARCHAR(20) NOT NULL,
    LName VARCHAR(20) NOT NULL,
    Sex CHAR(1) NOT NULL CHECK (Sex IN ('M', 'F')),
    Bdate DATE NOT NULL,
    Salary DECIMAL(10,2) NOT NULL CHECK (Salary >= 0),
      FOREIGN KEY (Mana_ID) REFERENCES Manager(Mana_ID)
  );

CREATE TABLE Manager (
    Mana_ID INT PRIMARY KEY,
    FName VARCHAR(20) NOT NULL,
    LName VARCHAR(20) NOT NULL,
    Salary DECIMAL(10,2) NOT NULL CHECK (Salary >= 0)
  );
    CREATE TABLE Driver_Location (
    EmployeeID INT,
    Location VARCHAR(100) NOT NULL,
    PRIMARY KEY (EmployeeID, Location),
    FOREIGN KEY (EmployeeID) REFERENCES Delivery_driver(EmployeeID)
  );
```

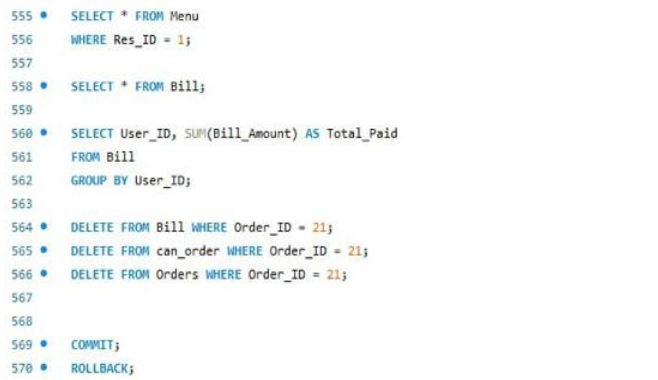


```
CREATE TABLE Restaurant (
    Res_ID INT PRIMARY KEY,
    Res_name VARCHAR(100) NOT NULL UNIQUE,
    Res_location VARCHAR(100) NOT NULL,
    phone_num VARCHAR(10) NOT NULL
  );

CREATE TABLE Oversees (
    Mana_ID INT PRIMARY KEY,
    Res_ID INT ,
    FOREIGN KEY (Mana_ID) REFERENCES Manager(Mana_ID),
    FOREIGN KEY (Res_ID) REFERENCES Restaurant(Res_ID)
  );

CREATE TABLE Menu (
    Menu_ID INT PRIMARY KEY,
    Food_name VARCHAR(100) NOT NULL,
    Food_price DECIMAL(10,2) NOT NULL CHECK (Food_price > 0),
    product_id INT NOT NULL UNIQUE,
    Res_ID INT,
    FOREIGN KEY (Res_ID) REFERENCES Restaurant(Res_ID)
  );
```



```
CREATE TABLE Users (
    User_ID INT PRIMARY KEY,
    FName VARCHAR(20) NOT NULL,
    LName VARCHAR(20) NOT NULL,
    Location VARCHAR(100) NOT NULL,
    user_phone VARCHAR(20) NOT NULL
);
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Order_Year INT NOT NULL CHECK (Order_Year BETWEEN 2000 AND 2100),
    Order_Month INT NOT NULL CHECK (Order_Month BETWEEN 1 AND 12),
    Order_Day INT NOT NULL CHECK (Order_Day BETWEEN 1 AND 31)
);
CREATE TABLE Handled_by (
    Order_ID INT,
    EmployeeID INT,
    PRIMARY KEY (Order_ID, EmployeeID),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
    FOREIGN KEY (EmployeeID) REFERENCES Delivery_driver(EmployeeID)
);
```

```
CREATE TABLE Bill (
    Bill_ID INT PRIMARY KEY,
    Bill_Amount DECIMAL(10,2) NOT NULL CHECK (Bill_Amount > 0),
    Order_ID INT,
    User_ID INT,
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
    FOREIGN KEY (User_ID) REFERENCES Users(User_ID)
  );

CREATE TABLE can_order (
    User_ID INT,
    Order_ID INT,
    PRIMARY KEY (User_ID, Order_ID),
    FOREIGN KEY (User_ID) REFERENCES Users(User_ID),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
  );


INSERT INTO Users (User_ID, FName, LName, Location, user_phone) VALUES
(21, 'Hassan', 'Alrashid', 'Riyadh', '530123456');

INSERT INTO Orders (Order_ID, Order_Year, Order_Month, Order_Day) VALUES
(21, 2025, 5, 5);
```



```
SELECT * FROM Menu
WHERE Res_ID = 1;

SELECT * FROM Bill;

SELECT User_ID, SUM(Bill_Amount) AS Total_Paid
FROM Bill
GROUP BY User_ID;

DELETE FROM Bill WHERE Order_ID = 21;
DELETE FROM can_order WHERE Order_ID = 21;
DELETE FROM Orders WHERE Order_ID = 21;


COMMIT;
ROLLBACK;
```

Functions:

```
DELIMITER //
CREATE FUNCTION GetOrderCount(p_UserID INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE order_count INT;
    SELECT COUNT(*) INTO order_count
    FROM can_order
    WHERE User_ID = p_UserID;
    RETURN order_count;
END //
DELIMITER ;

select projectdb.GetOrderCount(123);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

projectdb.GetOrderCount(123)
0

Stored procedures:

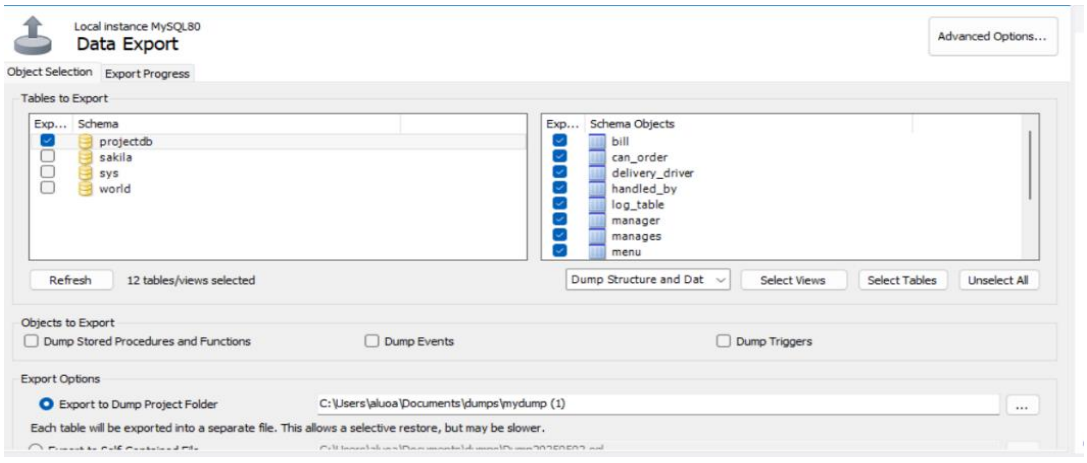
```
USE projectdb;
DROP FUNCTION IF EXISTS GetOrderCount;
USE projectdb;
DROP PROCEDURE IF EXISTS GetTotalBill;
DELIMITER //
CREATE PROCEDURE GetTotalBill(IN orderId INT)
BEGIN
    SELECT Bill_Amount FROM Bill WHERE Order_ID = orderId;
END //
DELIMITER ;
CALL GetTotalBill(1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

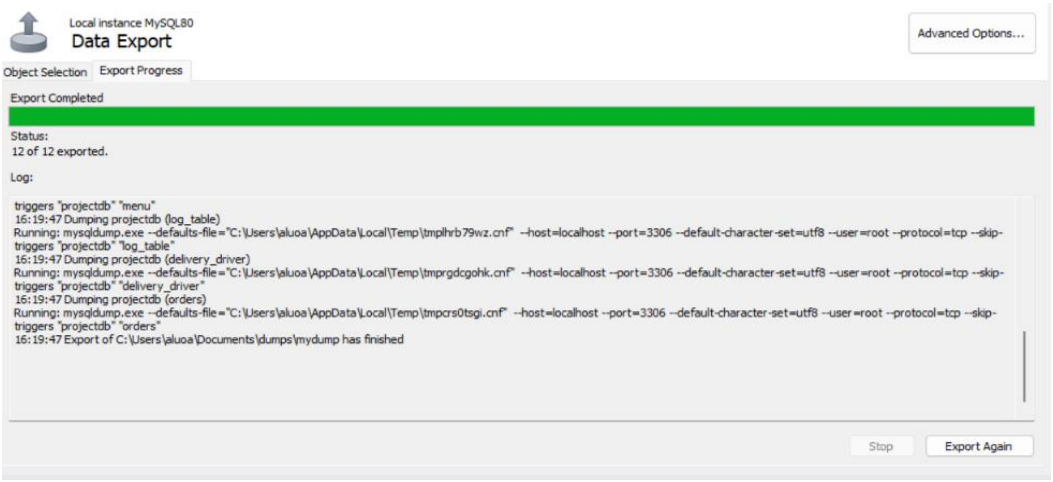
Bill_Amount
45.50

8. Back-up & Recovery

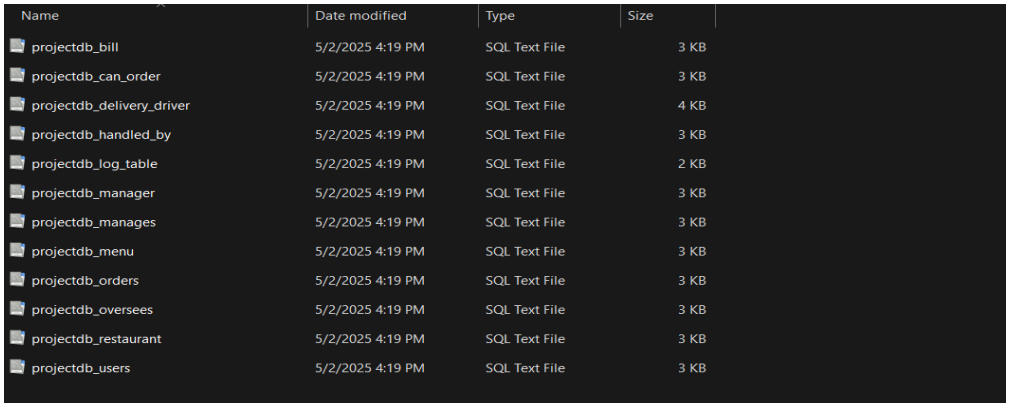
Used the Data Export tool in MySQL Workbench to back up the selected database and tables.



Export completed successfully and the backup files were saved in the specified folder.

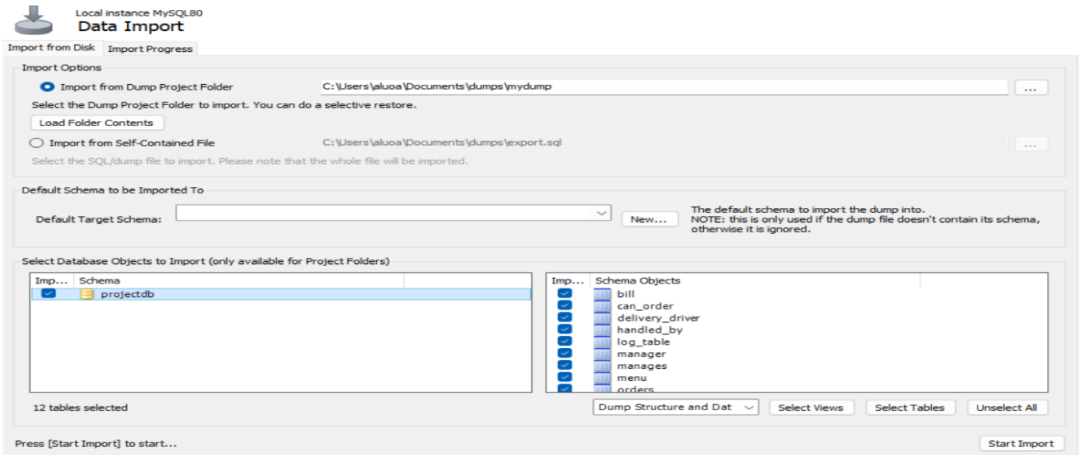


Backup SQL files are now available in the folder.

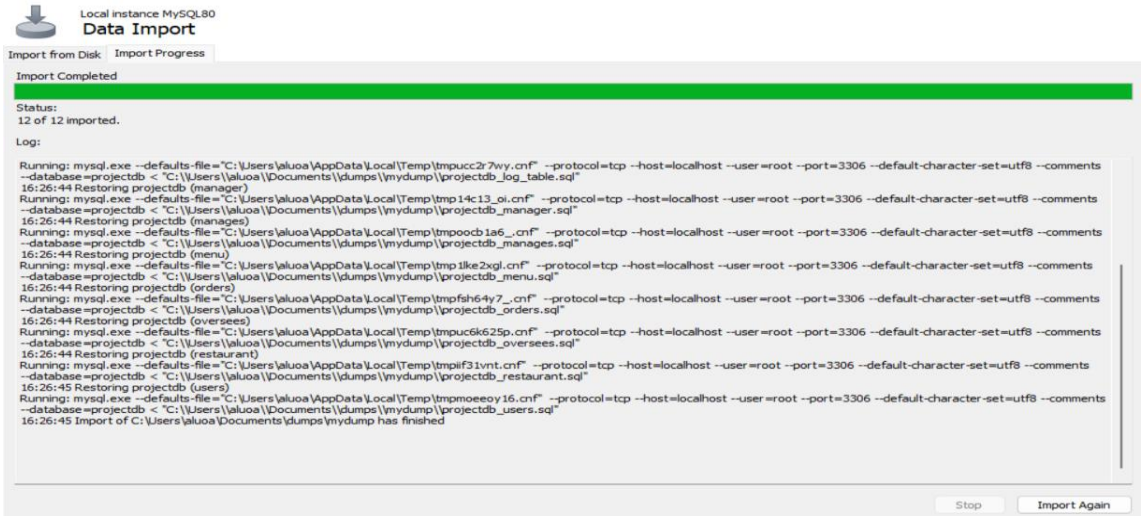


Name	Date modified	Type	Size
projectdb_bill	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_can_order	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_delivery_driver	5/2/2025 4:19 PM	SQL Text File	4 KB
projectdb_handled_by	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_log_table	5/2/2025 4:19 PM	SQL Text File	2 KB
projectdb_manager	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_manages	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_menu	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_orders	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_oversees	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_restaurant	5/2/2025 4:19 PM	SQL Text File	3 KB
projectdb_users	5/2/2025 4:19 PM	SQL Text File	3 KB

Selected the database and tables to import using the Data Import tool in MySQL Workbench



Successfully imported the selected database and tables. The process completed without errors.



9. Normalization

We performed normalization on our work, and in the end, we arrived at the following results:

1NF (First Normal Form)

• Each table has a primary key that uniquely identifies rows.

• All columns contain atomic (indivisible) values.

• No repeating groups or arrays in any table.

2NF (Second Normal Form)

• All non-key attributes in single-key tables are fully functionally dependent on the primary key.

• Composite-key tables like Driver_Location, Handled_by, and can_order do not have partial dependencies.

3NF (Third Normal Form)

• No transitive dependencies are observed within individual tables.

• Tables like Manager, Users, and Menu have non-key attributes that directly depend on the primary key.

• All foreign keys are used to maintain clear relational structure.